2.1: Various pointers

Aim: To study and implement different types of pointers and their applications in C programming for efficient memory management and data manipulation.

Program	Aim	Concept
1. Basic Pointer	To access the value and address of a variable using a pointer.	Basic Pointer
2. NULL Pointer	To demonstrate the use of a NULL pointer and verify it before dereferencing.	NULL Pointer
3. Wild Pointer	To illustrate an uninitialized pointer and understand its risks.	Wild Pointer
4. Dangling Pointer	To demonstrate a dangling pointer after memory deallocation and prevent it by assigning NULL.	Dangling Pointer
5. Pointer Arithmetic	To perform arithmetic operations on pointers for array traversal.	Pointer Arithmetic
6. Pointer to Pointer	To access a variable using a pointer to a pointer.	Pointer to Pointer
7. Void Pointer	To demonstrate the use of a generic (void) pointer with type casting.	Void Pointer
8. Constant Pointer	To demonstrate a constant pointer whose address cannot be changed.	Constant Pointer
9. Pointer to Constant	To demonstrate a pointer that cannot modify the value it points to.	Pointer to Constant
10. Constant Pointer to Constant	To demonstrate a pointer whose address and pointed value cannot be changed.	Constant Pointer to Constant
11. Function Pointer	To implement a function pointer for calling a function dynamically.	Function Pointer
12. Dynamic Memory Pointer (malloc)	To dynamically allocate memory for variables using malloc() and access the allocated memory.	Dynamic Memory Allocation
13. Array of Pointers	To create and access multiple variables using an array of pointers.	Array of Pointers
14. Pointer to Array	To access all elements of an array using a pointer to the entire array.	Pointer to Array

2.2: Dynamic Memory Allocation for an Array of Structures Using malloc()

Aim: To write a C program to dynamically allocate memory for an array of structures using the malloc() function, store details of multiple students, display the information, and release the allocated memory using free().

Algorithm

1. Start the program.
2. Define a structure Student containing Roll Number, Name, and Marks.
3. Declare a structure pointer.
4. Read the number of students (n).
5. Allocate memory dynamically for n students using malloc().
6. Check whether memory allocation is successful.
7. If allocation fails, display an error message and terminate.
8. Read the details of all students using a loop.
9. Display the stored student information.
10. Release the allocated memory using free().
11. Stop the program.

Sample Output

```
Enter Roll Number: 101
Enter Name: Rahul
Enter Marks: 89.5

Student Details
----------------
Roll Number : 101
Name        : Rahul
Marks       : 89.50
```

2.3: Dynamic Memory Allocation for Structure Using malloc()

Aim: To write a C program that dynamically allocates memory for a structure using the malloc() function, stores student information, displays the details, and releases the allocated memory using free().

Algorithm

1. Start the program.
2. Define a structure named Student containing roll number, name, and marks.
3. Declare a pointer of type struct Student.
4. Allocate memory dynamically using malloc(sizeof(struct Student)).
5. Check whether memory allocation is successful.
6. If allocation fails, display an error message and terminate the program.
7. Read the student's roll number, name, and marks.
8. Store the values in the dynamically allocated structure.
9. Display the stored student information.
10. Release the allocated memory using free().
11. Stop the program.

Expected Input and Output:

```
Enter Roll Number: 101
Enter Name: Rahul
Enter Marks: 89.5

Student Details
----------------
Roll Number : 101
Name        : Rahul
Marks       : 89.50
```

Experiment2.4: Dynamic Jagged Array (Rows of Different Sizes)

Aim: Write a C program to create a Dynamic Jagged Array, where each row can have a different number of elements, using dynamic memory allocation.

Algorithm

1. Read the number of rows.
2. Dynamically allocate memory for row pointers using malloc().
3. Read the size of each row.
4. Allocate memory for each row separately.
5. Read the elements of each row.
6. Display the jagged array.
7. Free the memory allocated for each row.
8. Free the memory allocated for row pointers.

Expected Input and Output:

```
Enter the number of rows: 3

Enter number of elements in Row 1: 4
Enter number of elements in Row 2: 2
Enter number of elements in Row 3: 5

Enter the elements:

Row 1:
10 20 30 40

Row 2:
50 60

Row 3:
70 80 90 100 110

Jagged Array:
 10  20  30  40
 50  60
 70  80  90 100 110

Memory released successfully.
```

Experiment 2.5: Mini Address Book using Dynamic Memory Allocation

Aim: Write a C program to implement a **Mini Address Book** using **dynamic memory allocation**. The program should support adding, displaying, searching, updating, and deleting contacts.

Algorithm

1. Start the program.
2. Dynamically allocate memory for contacts using realloc().
3. Display the menu.
4. Perform one of the following operations:
    - Add Contact
    - Display Contacts
    - Search Contact
    - Update Contact
    - Delete Contact
    - Exit
5. Free the allocated memory before exiting.

Expected Input and Output:

```
======================================
      MINI ADDRESS BOOK
======================================
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 1

Enter Name : Rahul
Enter Phone : 9876543210
Enter Email : rahul@gmail.com
Contact Added Successfully.

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 1

Enter Name : Priya
Enter Phone : 9123456789
Enter Email : priya@gmail.com
Contact Added Successfully.

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 2

------------------------------------------------------------
Name          Phone          Email
------------------------------------------------------------
Rahul         9876543210     rahul@gmail.com
Priya         9123456789     priya@gmail.com
```

```
======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 3

Enter Name to Search: Rahul

Contact Found
Name  : Rahul
Phone : 9876543210
Email : rahul@gmail.com

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 4

Enter Name to Update: Rahul
Enter New Phone : 9988776655
Enter New Email : rahul123@gmail.com
Contact Updated Successfully.

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 5

Enter Name to Delete: Priya
Contact Deleted Successfully.

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 2

------------------------------------------------------------
Name           Phone          Email
------------------------------------------------------------
Rahul          9988776655     rahul123@gmail.com

======================================
      MINI ADDRESS BOOK
======================================
Enter your choice: 6

Memory Released Successfully.
Exiting Program...
```

Experiment 2.6: Dynamic Voting System using Dynamic Memory Allocation

Aim: Write a C program to implement a **Dynamic Voting System** using **dynamic memory allocation**. The program should allow users to add candidates, cast votes, display results, search candidates, and determine the winner.

Algorithm

1. Start the program.
2. Allocate memory dynamically for candidate records using realloc().
3. Display the menu.
4. Perform one of the following operations:
    - Add Candidate
    - Cast Vote
    - Display Results
    - Search Candidate
    - Declare Winner
    - Exit
5. Free the allocated memory before exiting.

Expected input and output:

```
====================================
    DYNAMIC VOTING SYSTEM
====================================
1. Add Candidate
2. Cast Vote
3. Display Voting Results
4. Search Candidate
5. Declare Winner
6. Exit

Enter your choice: 1

Enter Candidate ID : 101
Enter Candidate Name : Alice
Candidate Added Successfully.

Enter your choice: 1

Enter Candidate ID : 102
Enter Candidate Name : Bob
Candidate Added Successfully.

Enter your choice: 2
Enter Candidate ID to Vote : 101
Vote Cast Successfully.

Enter your choice: 2
Enter Candidate ID to Vote : 101
Vote Cast Successfully.

Enter your choice: 2
Enter Candidate ID to Vote : 102
Vote Cast Successfully.

Enter your choice: 3
```

```
---------------------------------------------
ID      Candidate      Votes
---------------------------------------------
101     Alice          2
102     Bob            1

Enter your choice: 5

******** Election Result ********
Winner : Alice
Votes  : 2


Enter your choice: 4
Enter Candidate ID to Search : 104
Candidate Not Found.

=====================================
     DYNAMIC VOTING SYSTEM
=====================================
1. Add Candidate
2. Cast Vote
3. Display Voting Results
4. Search Candidate
5. Declare Winner
6. Exit
Enter your choice: 4
Enter Candidate ID to Search : 101

Candidate Found
ID    : 101
Name  : Alice
Votes : 2
```